

Taller 2 — Guía por fases y archivos modificados

Oct 2, 2026 · @Santiago Velásquez Holguín

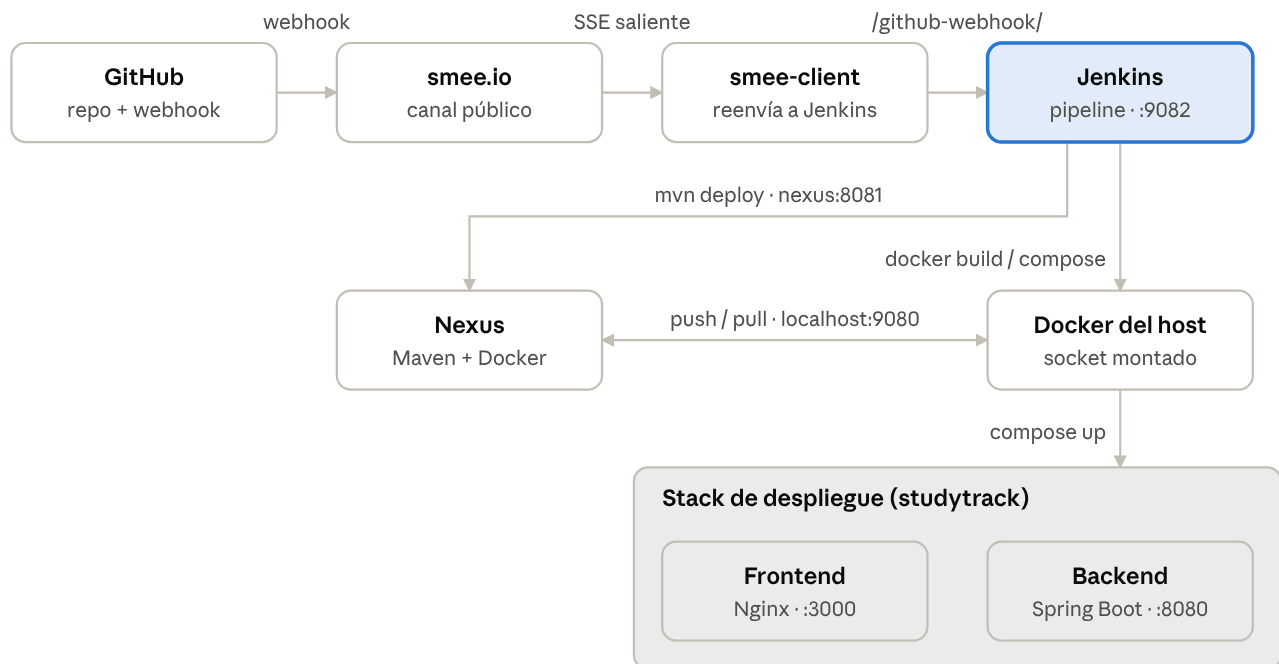
Resumen

Todos los TODO del código base están resueltos en el repo `Santiago-VH/taller-jenkins`; la raíz del proyecto es `codigo_base/` y Jenkins lee `codigo_base/Jenkinsfile`. Todo corre en contenedores sobre una sola máquina con Docker (sala de cómputo). La solución no se ha probado de punta a punta todavía: la primera ejecución real será en la sala.

Versionado inmutable: cada build publica `1.0.<BUILD_NUMBER>--<commit corto>` (ej. `1.0.7-a1b2c3d`) para el JAR, las dos imágenes y el despliegue. Nunca se usa `:latest`.

Dos direcciones para Nexus: `mvn deploy` corre dentro del contenedor de Jenkins y usa `http://nexus:8081`; `docker push/pull` lo ejecuta el daemon del host (socket montado) y usa `localhost:9080`.

Un push viaja por Smee hasta Jenkins, que publica en Nexus y despliega



ecosistema CI/CD · flujo de un push

Jenkins no tiene puertos expuestos a Internet: Smees entrega el webhook por una conexión saliente. El smoke test llega al backend porque este también está conectado a `cicd_network`.

Servicio	Host	Dentro de <code>cicd_network</code>
Jenkins UI	<code>http://localhost:9082</code>	<code>http://jenkins:8080</code>
Jenkins agentes	50000	—
Nexus UI / API	<code>http://localhost:9081</code>	<code>http://nexus:8081</code>
Nexus registry (docker-hosted)	<code>localhost:9080</code>	—
Backend	<code>http://localhost:8080/api/tasks</code>	<code>http://studytrack-api:8080</code>
Frontend	<code>http://localhost:3000</code>	—

Orden de ejecución: Fase 1 → Fase 2 (build local) → Fase 3 (Jenkins, luego Nexus, configuración, publicación manual) → Fase 4 (job, webhook, push).

Fase 1 — Pruebas y build

Esta fase no modifica archivos: solo confirma que el código base compila y pasa sus pruebas antes de containerizarlo.

Archivos modificados: ninguno.

Flujo de prueba

- Backend, desde `codigo_base/backend`: `mvn -B clean test`.
- Frontend, desde `codigo_base/frontend`: `npm install`, luego `npx tsc --noEmit` (tipos) y `npm run build` (genera `dist/`).
- Sin Maven o Node en el equipo, use contenedores (PowerShell):
 - Backend: `docker run --rm -v "${PWD}:/app" -w /app maven:3.9.6-eclipse-temurin-17-alpine mvn -B clean test`
 - Frontend: `docker run --rm -v "${PWD}:/app" -w /app node:20-alpine sh -c "npm install && npx tsc --noEmit && npm run build"`

Resultado esperado: `Tests run: 6` en `TaskServiceTest` y `Tests run: 5` en `TaskControllerTest`, ambos sin fallos; el build del frontend termina con `✓ built` y crea `dist/index.html`. El build del frontend ya se verificó; las pruebas del backend no se pudieron correr fuera de la sala.

Fase 2 — Empaquetamiento inmutable multi-stage

Ambas imágenes se construyen en dos etapas: la final no lleva compilador ni fuentes y corre con un usuario sin privilegios.

Archivos modificados

Archivo	Cambio
<code>backend/Dockerfile</code>	Builder <code>maven:3.9.6-eclipse-temurin-17-alpine</code> : copia <code>pom.xml</code> , <code>dependency:go-offline</code> (capa cacheable), luego <code>src</code> y <code>mvn package -DskipTests</code> . Runtime <code>eclipse-temurin:17-jre-alpine</code> , usuario <code>appuser</code> , puerto 8080. Recibe <code>ARG REVISION</code> (versión).
<code>frontend/Dockerfile</code>	Builder <code>node:20-alpine</code> : <code>npm install + npm run build</code> con <code>ARG VITE_API_URL</code> (por defecto <code>http://localhost:8080</code>). Runtime <code>nginx:1.27-alpine</code> , copia <code>dist/</code> , corre como usuario <code>nginx</code> en el puerto 3000.
<code>frontend/nginx.conf</code>	Fallback SPA: <code>try_files \$uri \$uri/ /index.html;</code>
<code>backend/pom.xml</code>	<code><version>\${revision}</version></code> (por defecto <code>1.0.0-local</code>), propiedad <code>nexus.url</code> (por defecto <code>http://localhost:9081</code>), <code>flatten-maven-plugin 1.6.0</code> y <code><distributionManagement></code> con id <code>nexus-releases</code> → <code>\${nexus.url}/repository/maven-releases/</code> .
<code>backend/ci-settings.xml</code> (nuevo)	<code>settings.xml</code> sin secretos: el servidor <code>nexus-releases</code> lee <code>\${env.NEXUS_USER}</code> y <code>\${env.NEXUS_PASS}</code> .

Archivo	Cambio
<code>backend/.dockerignore</code> , <code>frontend/.dockerignore</code> (nuevos)	Excluyen <code>target/</code> , <code>node_modules/</code> , <code>dist/</code> y <code>.flattened-pom.xml</code> del contexto de build.
<code>frontend/package-lock.json</code> (nuevo)	Fija las versiones exactas de las dependencias del frontend.

El enunciado nombra `maven:3.9.6-alpine`; se usó `maven:3.9.6-eclipse-temurin-17-alpine` porque es la etiqueta verificada en Docker Hub.

Flujo de prueba

1. Desde `codigo_base/backend`: `docker build -t studytrack-api:1.0.0-manual --build-arg REVISION=1.0.0-manual .`
2. Desde `codigo_base/frontend`: `docker build -t studytrack-frontend:1.0.0-manual .`
3. Revise tamaños con `docker images "studytrack*"` (backend cerca de 220 MB, frontend cerca de 50 MB).
4. Confirme el usuario no root: `docker run --rm --entrypoint whoami studytrack-api:1.0.0-manual` debe responder `appuser`.
5. Prueba rápida opcional: `docker run --rm -p 8080:8080 studytrack-api:1.0.0-manual` y abra `http://localhost:8080/api/tasks` (responde `[]`).
6. La publicación manual en Nexus que pide esta fase se hace cuando Nexus ya esté arriba (Fase 3, paso 4).

Fase 3 — Ecosistema CI/CD con Docker Compose

Se levantan dos stacks en la red compartida `cicd_network`: primero Jenkins + smee-client (crea la red) y después Nexus.

Archivos modificados

Archivo	Cambio
<code>infra/nexus_config/docker-compose.yml</code>	Puertos <code>9081:8081</code> (UI/API) y <code>9080:9080</code> (registry), volumen <code>nexus_data</code> , JVM limitada a 1 GB, healthcheck con <code>curl</code> a <code>/service/rest/v1/status</code> (start_period 180 s). Red <code>cicd_network</code> externa.
<code>infra/nexus_config/.env.example</code>	<code>NEXUS_UI_PORT=9081</code> , <code>NEXUS_DOCKER_REGISTRY_PORT=9080</code> .
<code>infra/jenkins_config/docker-compose.yml</code>	Servicio <code>jenkins</code> : user: <code>root</code> , puertos <code>9082:8080</code> y <code>50000:50000</code> , volumen <code>jenkins_home</code> , socket <code>/var/run/docker.sock</code> , healthcheck a <code>/login</code> . Servicio <code>smee</code> : <code>SMEE_URL</code> desde <code>.env</code> , <code>SMEE_TARGET=http://jenkins:8080/github-webhook/</code> , <code>depends_on jenkins healthy</code> . Crea la red <code>cicd_network</code> .
<code>infra/jenkins_config/.env.example</code>	<code>JENKINS_UI_PORT=9082</code> , <code>JENKINS_AGENT_PORT=50000</code> , <code>SMEE_CHANNEL_URL</code> .
<code>infra/jenkins_config/Dockerfile</code>	Instala Docker CLI + plugin compose (repo oficial de Docker) y Apache Maven 3.9.9.
<code>infra/jenkins_config/plugins.txt</code>	Agrega <code>github:1.37.3.1</code> , que crea el endpoint <code>/github-webhook/</code> y el trigger del job.
<code>infra/jenkins_config/smee/Dockerfile</code>	Variables por defecto <code>SMEE_URL</code> / <code>SMEE_TARGET</code> y usuario <code>node</code> (sin root).
<code>.gitignore</code>	Ignora <code>.env</code> , <code>target/</code> , <code>node_modules/</code> , <code>dist/</code> , <code>.flattened-pom.xml</code> , <code>*.tsbuildinfo</code> .

Flujo de prueba

1. Cree un canal en <https://smee.io> y copie su URL.

2. Jenkins, desde `codigo_base/infra/jenkins_config: cp .env.example .env` (ponga su `SREE_CHANNEL_URL`), `docker compose up -d --build`, `docker compose ps` hasta ver `healthy`. Clave inicial: `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`.
3. Nexus, desde `codigo_base/infra/nexus_config: cp .env.example .env`, `docker compose up -d`, esperar `healthy` (3-5 min). Clave: `docker exec nexus cat /nexus-data/admin.password`.
4. En `http://localhost:9081`:
 - Inicie sesión como `admin`, defina clave nueva y desactive el acceso anónimo.
 - Settings → Security → Realms: active **Docker Bearer Token Realm**.
 - Create repository → docker (hosted): nombre `docker-hosted`, **HTTP 9080**, Deployment policy **Disable redeploy**.
 - Revise que `maven-releases` tenga Version policy Release y Disable redeploy.
5. Publicación manual (cierra la Fase 2), desde `codigo_base/backend` en PowerShell:
 - `$env:NEXUS_USER="admin"; $env:NEXUS_PASS("<clave>")`
 - `mvn -B deploy -DskipTests -s ci-settings.xml -Drevision=1.0.0-manual`
 - `docker login localhost:9080 -u admin`, `docker tag studytrack-api:1.0.0-manual localhost:9080/studytrack-api:1.0.0-manual`, `docker push localhost:9080/studytrack-api:1.0.0-manual`
6. En `http://localhost:9082`: instale los plugins sugeridos, cree su usuario y registre las credenciales (Manage Jenkins → Credentials → Global): `nexus-credentials` (usuario y clave de Nexus) y `github-credentials` (usuario de GitHub + token, solo si el repo es privado).

Resultado esperado: `docker ps` muestra `jenkins`, `nexus` y `sree-client` arriba, los dos primeros `healthy`; Nexus → Browse muestra `studytrack-api/1.0.0-manual` en `maven-releases` y en `docker-hosted`; `docker logs sree-client` muestra el reenvío hacia `http://jenkins:8080/github-webhook/`.

Fase 4 — Pipeline declarativo en Jenkins

Un `git push` llega por `sree.io` a Jenkins, que prueba, empaqueta, publica en Nexus y despliega sin intervención manual.

Archivos modificados

Archivo	Cambio
<code>codigo_base/Jenkinsfile</code>	Trigger <code>githubPush()</code> ; opciones <code>timestamps</code> , <code>disableConcurrentBuilds</code> , <code>buildDiscarder</code> , <code>timeout 30 min</code> ; variables <code>APP_DIR</code> , <code>NEXUS_REGISTRY=localhost:9080</code> , <code>NEXUS_URL=http://nexus:8081</code> , <code>NEXUS_CREDENTIALS_ID=nexus-credentials</code> ; las 4 etapas; <code>docker logout</code> al final.
<code>codigo_base/deploy/docker-compose.yml</code>	Proyecto <code>studytrack</code> . <code>backend</code> y <code>frontend</code> con imagen <code>\${NEXUS_REGISTRY}/<servicio>:\${IMAGE_TAG}</code> (falla si falta <code>IMAGE_TAG</code>), puertos 8080 y 3000. Backend en <code>product-network</code> y <code>cicd_network</code> , healthcheck <code>wget a /api/tasks</code> . Frontend espera <code>service_healthy</code> del backend.

Qué hace cada etapa

- Checkout & Test:** `checkout scm`, calcula `IMAGE_TAG = 1.0.<BUILD_NUMBER>-<commit corto>`, renombra el build y corre `mvn -B clean test`; publica el reporte JUnit.
- Package & Tag Inmutable:** `mvn package -Drevision=$IMAGE_TAG` y `docker build` del backend y el frontend con el tag inmutable y la etiqueta OCI del commit.
- Publish to Nexus:** dentro de `withCredentials`, `mvn deploy -s ci-settings.xml` hacia `maven-releases`, luego `docker login` y `docker push` de las dos imágenes a `docker-hosted`.
- Deploy & Smoke Test:** `docker compose pull + up -d` del stack de `deploy/`, luego `curl a http://studytrack-api:8080/api/tasks` con hasta 20 intentos cada 5 s. Si pasa, guarda el tag en `$JENKINS_HOME/studytrack-last-good-tag`; si falla, muestra logs y hace rollback a ese tag.

Flujo de prueba

- En Jenkins: New Item → `studytrack-pipeline` (Pipeline). Marque **GitHub hook trigger for GITScm polling**. Definition: Pipeline script from SCM → Git, URL `https://github.com/Santiago-VH/taller-jenkins.git`, rama `*/main`, credencial `github-credentials` si el repo es privado, **Script Path** `codigo_base/Jenkinsfile`.
- Ejecute **Build Now** una vez: registra el trigger y valida las 4 etapas.

3. En GitHub → Settings → Webhooks → Add webhook: Payload URL = su canal de smee.io, Content type `application/json`, solo evento push.
4. Haga un commit cualquiera y `git push`.

Resultado esperado: el build arranca solo con *Started by GitHub push*; Stage View muestra las 4 etapas en verde; el build se llama `#N - 1.0.N-<commit>`; Nexus tiene el JAR y las dos imágenes con ese tag; `http://localhost:3000` muestra la app y `http://localhost:8080/api/tasks` responde.

Entrega y solución de problemas

`GUIA_TALLER.md` (raíz del repo) repite estos pasos en formato de terminal para tenerlos a mano en la sala; se puede borrar antes de entregar.

Evidencias para el PDF de resultados

- ☐ Enlace al repositorio privado con acceso al docente
- ☐ Captura del Stage View con las 4 etapas en verde
- ☐ Evidencia del disparo automático: *Started by GitHub push* y la entrega en smee.io o en `docker logs smee-client`
- ☐ Capturas de `maven-releases` y `docker-hosted` en Nexus con el artefacto publicado
- ☐ Cuestionario, autorreflexión (mínimo 150 palabras) y bitácora de prompts IAG

Problemas frecuentes

Síntoma	Solución
<code>network cicd_network ... could not be found</code>	Levante primero el stack de Jenkins o ejecute <code>docker network create cicd_network</code> .
<code>jenkins-plugin-cli</code> falla por versiones de dependencias	Quite los números de versión en <code>plugins.txt</code> y reconstruya con <code>docker compose build --no-cache</code> .
<code>permission denied ... docker.sock</code>	El servicio jenkins debe tener <code>user: root</code> (ya está en el compose).
<code>docker login localhost:9080</code> da 401 o conexión rechazada	Active Docker Bearer Token Realm y revise que <code>docker-hosted</code> use HTTP 9080.

Síntoma	Solución
<code>mvn deploy</code> da 400 Bad Request	La versión ya existe (Disable redeploy) o es <code>-SNAPSHOT</code> .
El push no dispara el build	Revise <code>docker logs smee-client</code> , la casilla del trigger y que el job haya corrido una vez.
Puerto ocupado	Cámbielo en el <code>.env</code> ; si es 9080 o 9081, ajuste también <code>NEXUS_REGISTRY</code> en el Jenkinsfile y <code>nexus.url</code> en el <code>pom.xml</code> .